



UNIDADE IV

Análise e Projeto de Sistemas

Profa. MSc. Cristiane Fidelix

Capítulo 7 - Projeto de dados e projeto arquitetural

Índice: 1ª parte: Neste capítulo veremos:

- Uma breve introdução ao projeto de dados.
- Introdução a banco de dados relacionais.
- **Visualizar exemplos de bancos relacionais utilizando:** MER (modelo de entidade e relacionamento), DER e modelo físico.

Capítulo 7 - Projeto de dados e projeto arquitetural

Introdução ao projeto de dados

- O projeto de dados, ou modelagem de dados, tem como objetivo definir uma estrutura de informações necessárias para implementar o sistema de software (Pressman, 2006).
- Como vimos, boa parte dessas atividades é feita pelo SGBD, que é o responsável por prover os mecanismos de acesso, gerenciamento e processamento das informações em um repositório de dados.

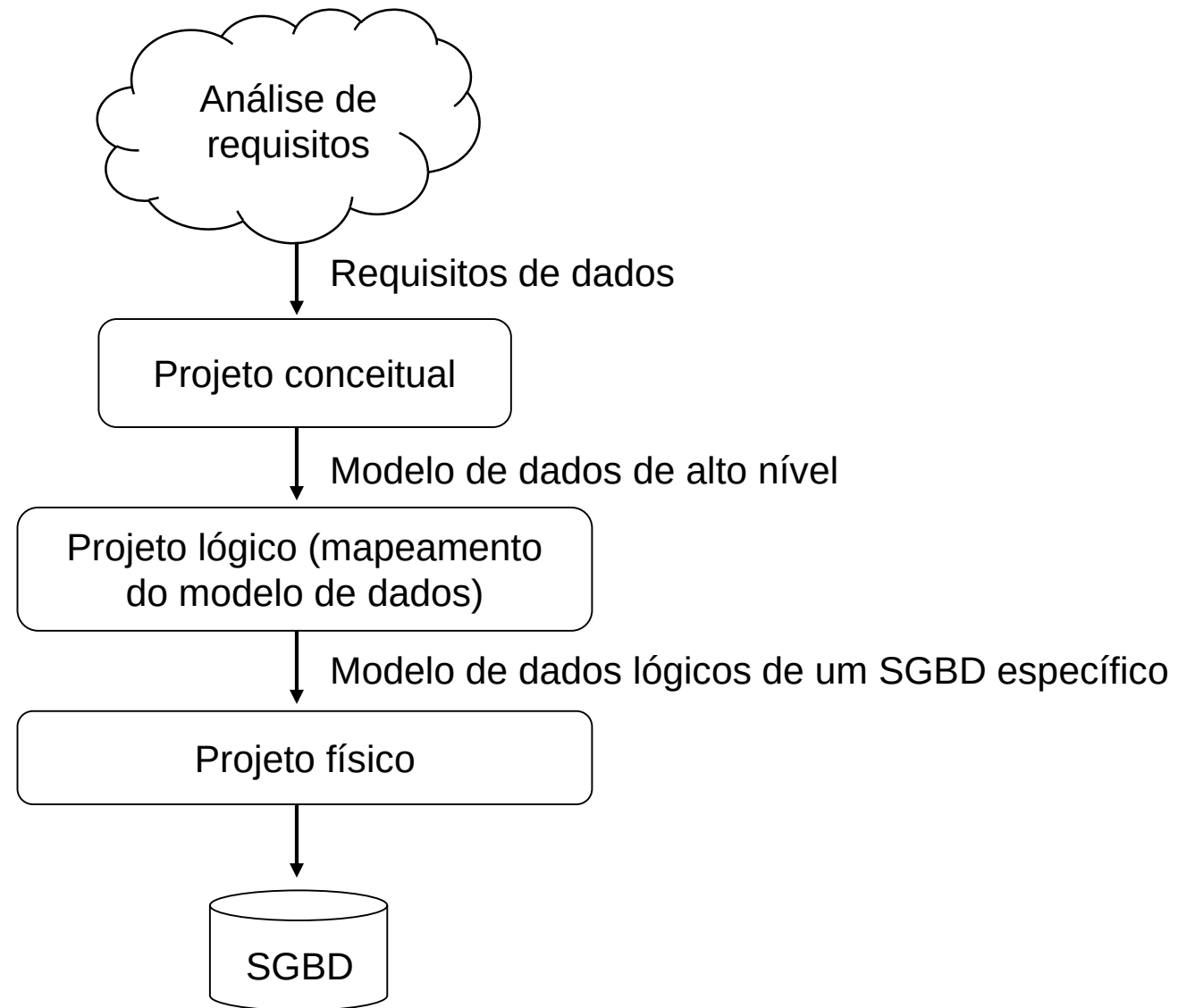
Capítulo 7 - Projeto de dados e projeto arquitetural

Introdução ao projeto de dados

- Projetar um banco de dados também não é uma tarefa que pode ser realizada sem organização.
- **O processo de projeto de banco de dados pode ser subdividido em três fases: projeto conceitual, projeto lógico e projeto físico, como mostra a figura a seguir (Elmasri; Navathe, 2011).**

Capítulo 7 - Projeto de dados e projeto arquitetural

Banco de dados



Capítulo 7 - Projeto de dados e projeto arquitetural

Introdução a banco de dados relacionais

O que é um banco de dados relacional?

- Um Banco de Dados Relacional (BDR) é um sistema que armazena dados em tabelas organizadas em linhas e colunas.
- Utiliza o modelo relacional, proposto por Edgar F. Codd em 1970.

Capítulo 7 - Projeto de dados e projeto arquitetural

Introdução a banco de dados relacionais: Características principais

- **Tabelas (relações):** Estruturas que armazenam os dados.
- **Chaves primárias:** Identificam unicamente cada linha de uma tabela.
- **Chaves estrangeiras:** Criam relações entre tabelas diferentes.

Capítulo 7 - Projeto de dados e projeto arquitetural

Introdução a banco de dados relacionais: Características principais

- **Consultas com SQL:** Linguagem padrão para manipulação dos dados.
- **Exemplo:** SELECT, INSERT, UPDATE.

Capítulo 7 - Projeto de dados e projeto arquitetural

Introdução a banco de dados relacionais: Vantagens

- **Integridade dos dados:** Regras como chave primária evitam duplicações.
 - **Flexibilidade nas consultas:** SQL permite filtros, junções e ordenações.
 - **Segurança:** Controle de acesso por usuários.
 - Escalabilidade e consistência.
-
- **Principais SGBDs relacionais:** MySQL, PostgreSQL, Oracle, SQL Server, SQLite.

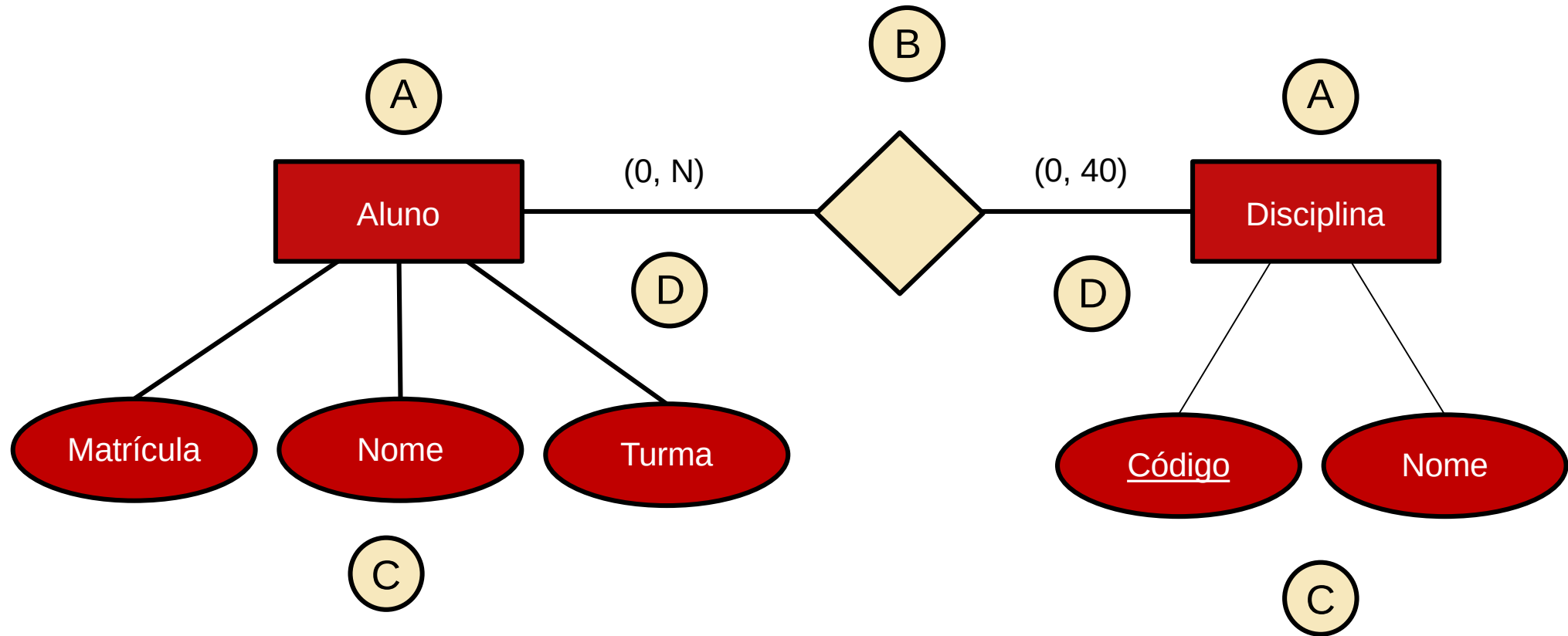
Capítulo 7 - Projeto de dados e projeto arquitetural

O modelo entidade relacional

- **Enxerga os dados do mundo real como o conjunto:** entidade, atributos e relacionamento.
- Cada entidade, ou um conjunto de entidades, gera uma tabela; seus atributos ou características são representados por colunas desta tabela e cada linha representa uma instância dessa entidade.

Capítulo 7 - Projeto de dados e projeto arquitetural

- Modelo conceitual (MER), de alto nível, produzido na fase conceitual do projeto de banco de dados.



Capítulo 7 - Projeto de dados e projeto arquitetural

- Modelo lógico: DER



Capítulo 7 - Projeto de dados e projeto arquitetural

Modelo físico:

- Estruturas das tabelas

MATRÍCULA	NOME	TURMA
12345	Maria de Maria	ADS3
54321	João da Silva	ADS3
667788	Pedro de Pietro	ADS4

CÓDIGO	NOME
BD001	Banco de Dados
POO1	Programação Orientada a Objetos 1
WEB01	Desenvolvimento Web

MATRÍCULA	CÓDIGO
12345	BD001
54321	BD001
667788	BD001
12345	POO1
54321	P001
667788	WEB01

Fonte: adaptado de: livro-texto.

Capítulo 7 - Projeto de dados e projeto arquitetural

Modelo físico:

- Implementação a partir da linguagem SQL

```
CREATE TABLE ALUNO
(
    MATRICULA INT NOT NULL,
    NOME VARCHAR(20) NOT NULL,
    TURMA VARCHAR(10) NOT NULL,
    PRIMARY KEY (MATRICULA)
)

CREATE TABLE DISCIPLINA
(
    CODIGO VARCHAR(20) NOT NULL,
    NOME VARCHAR(100) NOT NULL,
    PRIMARY KEY (CODIGO)
)

CREATE TABLE ALUNO_DISCIPLINA
(
    MATRICULA_ALUNO,
    CODIGO_DISCIPLINA,
    FOREIGN KEY (MATRICULA_ALUNO)
        REFERENCES ALUNO(MATRICULA),
    FOREIGN KEY (CODIGO_DISCIPLINA)
        REFERENCES DISCIPLINA(CÓDIGO)
)
```

Interatividade

Assinale a alternativa que melhor descreve uma característica fundamental dos bancos de dados relacionais:

- a) Os dados são armazenados em arquivos de texto simples e organizados em parágrafos.
- b) A principal forma de organização dos dados é por meio de tabelas relacionadas e identificação de chaves primárias.
- c) As consultas são feitas exclusivamente por meio de menus gráficos.
- d) Os bancos relacionais não permitem criar relacionamentos entre diferentes conjuntos de dados.
- e) Cada tabela pode conter um tipo de dado, como números inteiros ou texto, nunca ambos.

Resposta

Assinale a alternativa que melhor descreve uma característica fundamental dos bancos de dados relacionais:

- a) Os dados são armazenados em arquivos de texto simples e organizados em parágrafos.
- b) A principal forma de organização dos dados é por meio de tabelas relacionadas e identificação de chaves primárias.
- c) As consultas são feitas exclusivamente por meio de menus gráficos.
- d) Os bancos relacionais não permitem criar relacionamentos entre diferentes conjuntos de dados.
- e) Cada tabela pode conter um tipo de dado, como números inteiros ou texto, nunca ambos.

Capítulo 7 - Projeto de dados e projeto arquitetural

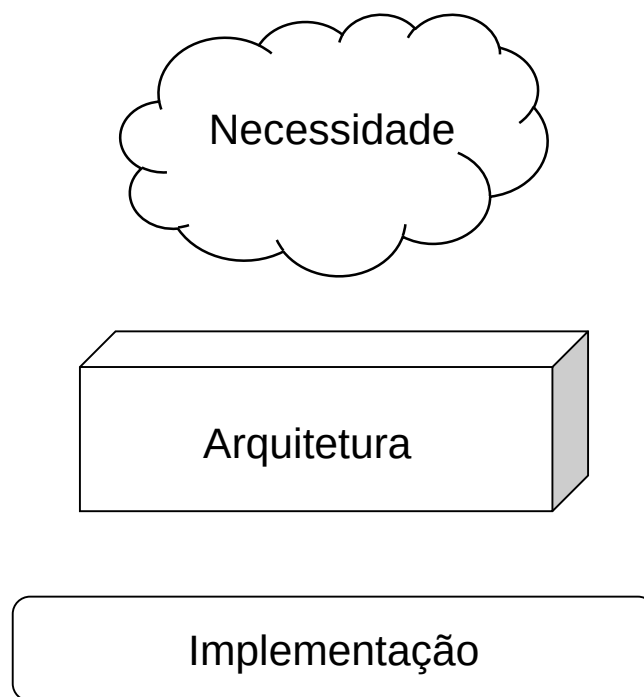
Índice: 2ª parte

- Qual é o papel da arquitetura? O que é o projeto arquitetural?
- A importância da arquitetura de software.
- Processo de arquitetura de software em aspectos humanos.
- As fases do processo de arquitetura.

Capítulo 7 - Projeto de dados e projeto arquitetural

Qual é o papel da arquitetura?

- Na engenharia de software, o papel da arquitetura é bem semelhante ao da arquitetura na engenharia civil, ou seja, a projeção de estruturas, a constituição de um software ou a contextura de um todo, assim como na arquitetura da engenharia civil.



Capítulo 7 - Projeto de dados e projeto arquitetural

A importância da arquitetura de software

- **É a mesma importância que a arquitetura tem na construção de uma casa:**
a arquitetura, ou o projeto arquitetural, representado por uma planta, por exemplo, é o principal guia para a construção.

Capítulo 7 - Projeto de dados e projeto arquitetural

A importância da arquitetura de software

- Facilita a comunicação entre os envolvidos no projeto.
- Representa, em escala menor e compreensível, a forma pela qual os componentes de um sistema são estruturados e interagem.
- Modela e coloca em evidência as decisões de projeto como fator fundamental para o sucesso do sistema de software.

Capítulo 7 - Projeto de dados e projeto arquitetural

Importância:

- Facilita a evolução do software.
- Facilita o reúso de componentes, logo, promove maior produtividade na construção e na manutenção, pois também facilita a manutenção.
- Promove a diminuição do espaço entre as fases de análise e construção e, por consequência, a diminuição do retrabalho na construção.

Capítulo 7 - Projeto de dados e projeto arquitetural

Importância:

- Auxilia a gestão do projeto quanto à estimativa de tempo, ao custo e à complexidade do projeto.
- Promove mitigação e diminuição de riscos.

Capítulo 7 - Projeto de dados e projeto arquitetural

Processo de arquitetura de software

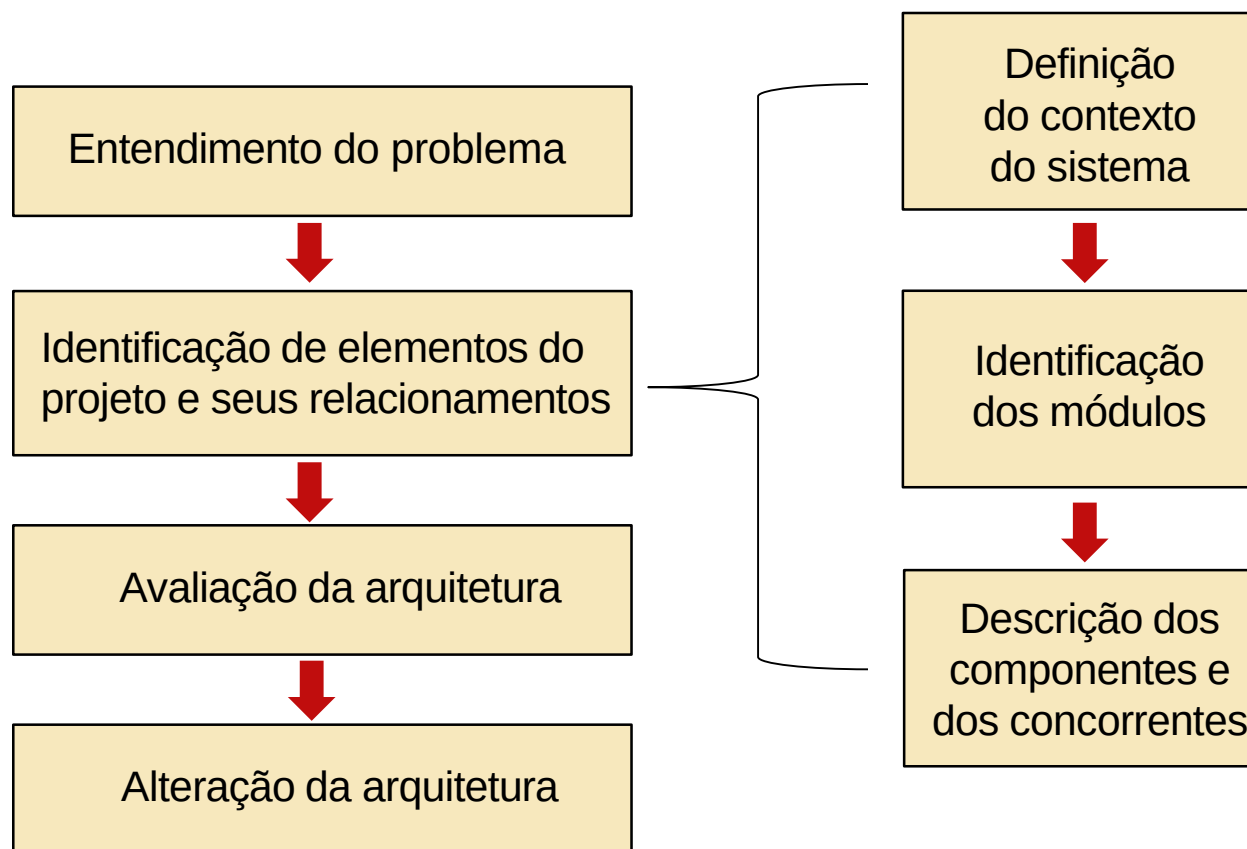
Segundo Albin (2003), a participação de arquitetura de software pode ser dividida em cinco fases, e deve ser iniciada antes mesmo da fase de projetos do ciclo de vida da engenharia de software:

Fases do ciclo de vida da engenharia de software		Fases da arquitetura
Análise de requisitos	➔	Pré-projeto
Projeto	➔	Análise de domínio
		Projeto esquemático
Implementação e testes	➔	Desenvolvimento de projeto
		Construção
Implantação	➔	(Sem correspondência)

Capítulo 7 - Projeto de dados e projeto arquitetural

Processo de arquitetura de software

- A figura a seguir mostra a sequência de atividades do processo de definição de arquitetura sugerido por Albin (2003), e o quadro na sequência contextualiza o objetivo de cada fase.



Fonte: adaptada de: livro-texto.

Capítulo 7 - Projeto de dados e projeto arquitetural

Fases do processo de arquitetura

- **Entendimento do problema:** A fase de entendimento do problema deve ser iniciada, idealmente, na fase de análise de requisitos, como dissemos anteriormente.
- **Identificação de elementos do projeto e seus relacionamentos:** Definição do contexto do sistema, identificação dos módulos e descrição dos componentes e dos conectores.

Capítulo 7 - Projeto de dados e projeto arquitetural

Fases do processo de arquitetura

- **Avaliação da arquitetura:** A atividade de avaliação da arquitetura tem como objetivo verificar possíveis pontos frágeis da arquitetura, para melhoria.
- **Alteração da arquitetura:** A alteração da arquitetura vem em decorrência dos possíveis pontos frágeis encontrados na fase de avaliação.

Capítulo 7 - Projeto de dados e projeto arquitetural

Processo de arquitetura de software em aspectos humanos

- O processo de arquitetura de software e as demais atividades desenvolvidas ao longo de todo o ciclo de desenvolvimento do projeto possuem alta dependência do fator humano.

Capítulo 7 - Projeto de dados e projeto arquitetural

Processo de arquitetura de software em aspectos humanos

Segundo o OpenUP ([s.d.]), existem alguns interessados no processo de definição de arquitetura de um sistema de software:

- Analistas de requisitos, desenvolvedores, gerentes de projetos e arquitetos de software.

Capítulo 7 - Projeto de dados e projeto arquitetural

Processo de arquitetura de software em aspectos humanos

- Quem é o arquiteto de softwares?

Capítulo 7 - Projeto de dados e projeto arquitetural

Processo de arquitetura de software em aspectos humanos

- Conhecer os aspectos culturais do seu cliente.
- Conhecer as estratégias de mercado do cliente e o que ele pretende com o software.
- Conhecer as limitações de tecnologia e infraestrutura do cliente.
- Conhecer o domínio do negócio e do problema sobre o qual pretende promover uma solução.

Capítulo 7 - Projeto de dados e projeto arquitetural

Processo de arquitetura de software em aspectos humanos

- As responsabilidades do arquiteto no projeto arquitetural?

Capítulo 7 - Projeto de dados e projeto arquitetural

Processo de arquitetura de software em aspectos humanos

- Converter os requisitos do cliente em um projeto de software.
- Promover um estudo sobre a viabilidade das soluções técnicas adotadas, a chamada prova de conceitos.

Capítulo 7 - Projeto de dados e projeto arquitetural

Processo de arquitetura de software em aspectos humanos

- Avaliar as tendências tecnológicas, em vista das soluções adotadas no projeto, sob o ponto de vista de competitividade do produto a ser desenvolvido.
- Buscar o aumento do tempo de vida útil do software.

Interatividade

Qual das alternativas abaixo não descreve completamente as fases do ciclo de vida da engenharia de software, segundo Albin?

- a) Análise de requisitos => Pré-Projeto.
- b) Análise de requisitos => Análise de Domínio.
- c) Projeto => Análise de Domínio e Projeto Esquemático.
- d) Implementação e Testes => Desenvolvimento do Projeto e Construção.
- e) Implantação => sem correspondência.

Resposta

Qual das alternativas abaixo não descreve completamente as fases do ciclo de vida da engenharia de software, segundo Albin?

- a) Análise de requisitos => Pré-Projeto.
- b) Análise de requisitos => Análise de Domínio.
- c) Projeto => Análise de Domínio e Projeto Esquemático.
- d) Implementação e Testes => Desenvolvimento do Projeto e Construção.
- e) Implantação => sem correspondência.

Capítulo 8 - Visões da arquitetura de software

Índice: 3ª parte

Vimos no capítulo anterior sobre projeto arquitetural e sua importância. Neste capítulo, vamos abordar:

- A utilização de padrões na arquitetura de software.

Capítulo 8 - Visões da arquitetura de software

Índice: 3ª parte

- Uma breve introdução a padrões de projeto - Design patterns.
- Estruturação de sistemas em subsistemas e camadas e a importância da arquitetura de software.

Capítulo 8 - Visões da arquitetura de software

A importância da arquitetura de software

- É a mesma importância que a arquitetura tem na construção de uma casa.
- Arquitetura, ou o projeto arquitetural, representado por uma planta, por exemplo, é o principal guia para a construção.

Capítulo 8 - Visões da arquitetura de software

Vimos qual a importância da arquitetura de software

- Facilita a comunicação entre os envolvidos no projeto.
- Representa, em escala menor e compreensível, a forma pela qual os componentes de um sistema são estruturados e interagem.
- Modela e coloca em evidência as decisões de projeto como fator fundamental para o sucesso do sistema de software.

Capítulo 8 - Visões da arquitetura de software

Vimos qual a importância da arquitetura de software

- O processo de arquitetura de software e as demais atividades desenvolvidas ao longo de todo o ciclo de desenvolvimento do projeto possuem alta dependência do fator humano.

Capítulo 8 - Visões da arquitetura de software

Visões da arquitetura de software

- **Esses componentes podem ser divididos em dois grupos:** estáticos e dinâmicos.

Capítulo 8 - Visões da arquitetura de software

Visão estática:

- Promove a visão da organização dos componentes do software e de suas relações com elementos de dados (banco de dados, arquivos-texto etc.), com sistemas de hardware e com outros sistemas de software.

Capítulo 8 - Visões da arquitetura de software

A visão dinâmica

- Promove a visão comportamental do sistema e de seus componentes durante a execução de uma determinada ação.
- Nessa visão, definimos como os componentes do sistema reagem a eventos internos e externos e ainda a forma como eles se comunicam entre si, ou seja, o protocolo de comunicação interno, e até mesmo como eles se comunicam com componentes externos ao domínio do software, chamado de protocolo de comunicação externa.

Capítulo 8 - Visões da arquitetura de software

Por que utilizar padrões?

- Utilização de padrões na arquitetura de software:

Utilização de padrões e padronização é um conceito amplamente utilizado por outras engenharias.

Capítulo 8 - Visões da arquitetura de software

Utilização de padrões

- A engenharia civil, por exemplo, possui uma gama de padrões para resolver o problema de distribuição elétrica de uma residência, assim como a engenharia elétrica possui um conjunto de soluções-padrão para a comunicação entre componentes eletrônicos em uma placa de circuito.

Capítulo 8 - Visões da arquitetura de software

Utilização de padrões

Categoria	Estilo arquitetural
From mud to structure (Da "lama" à estrutura)	Camadas ou Layers
	Pipes and Filters
	Blackboard
Sistemas distribuídos	Broker
	Microkernel
	Pipes and Filters
Sistemas interativos	Model-View-Controller (MVC)
	Presentation-Abstraction-Control (PAC)
Sistemas adaptáveis	Reflection
	Microkernel

Fonte: adaptado de: livro-texto.

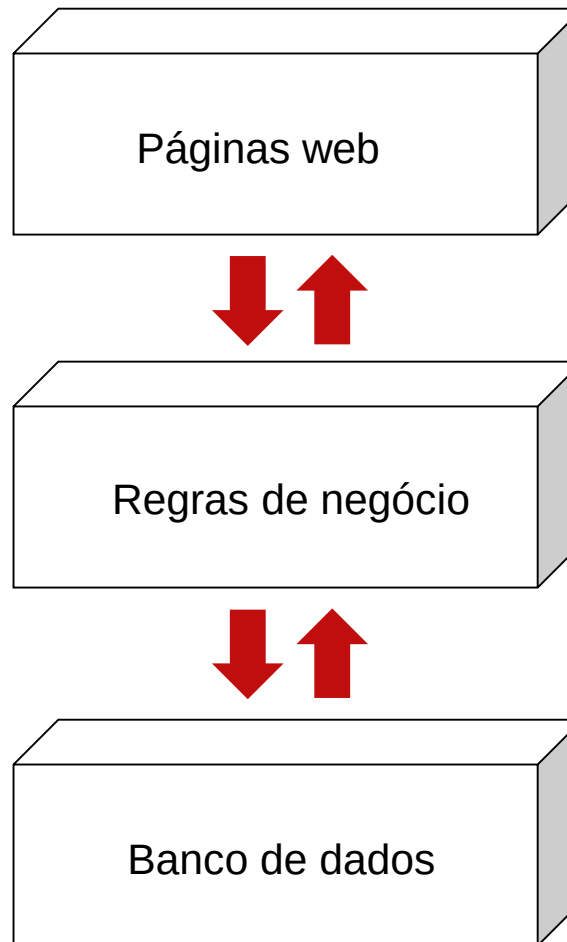
Capítulo 8 - Visões da arquitetura de software

Estruturação de sistemas em subsistemas e camadas

- **Imagine a seguinte situação:** você é contratado para desenvolver um simples sistema de cadastro de funcionários. Nesse sistema, terá apenas de inserir informações básicas de um funcionário, como nome, CPF, RG, número do PIS e salário; no requisito salário, o usuário do sistema entra com o salário bruto e o sistema automaticamente calcula o salário líquido a partir de algumas regras, como desconto de imposto de renda e INSS. Suponhamos ainda que esse sistema seja um sistema web.

Capítulo 8 - Visões da arquitetura de software

Estruturação de sistemas em subsistemas e camadas



Fonte: adaptado de: livro-texto.

Capítulo 8 - Visões da arquitetura de software

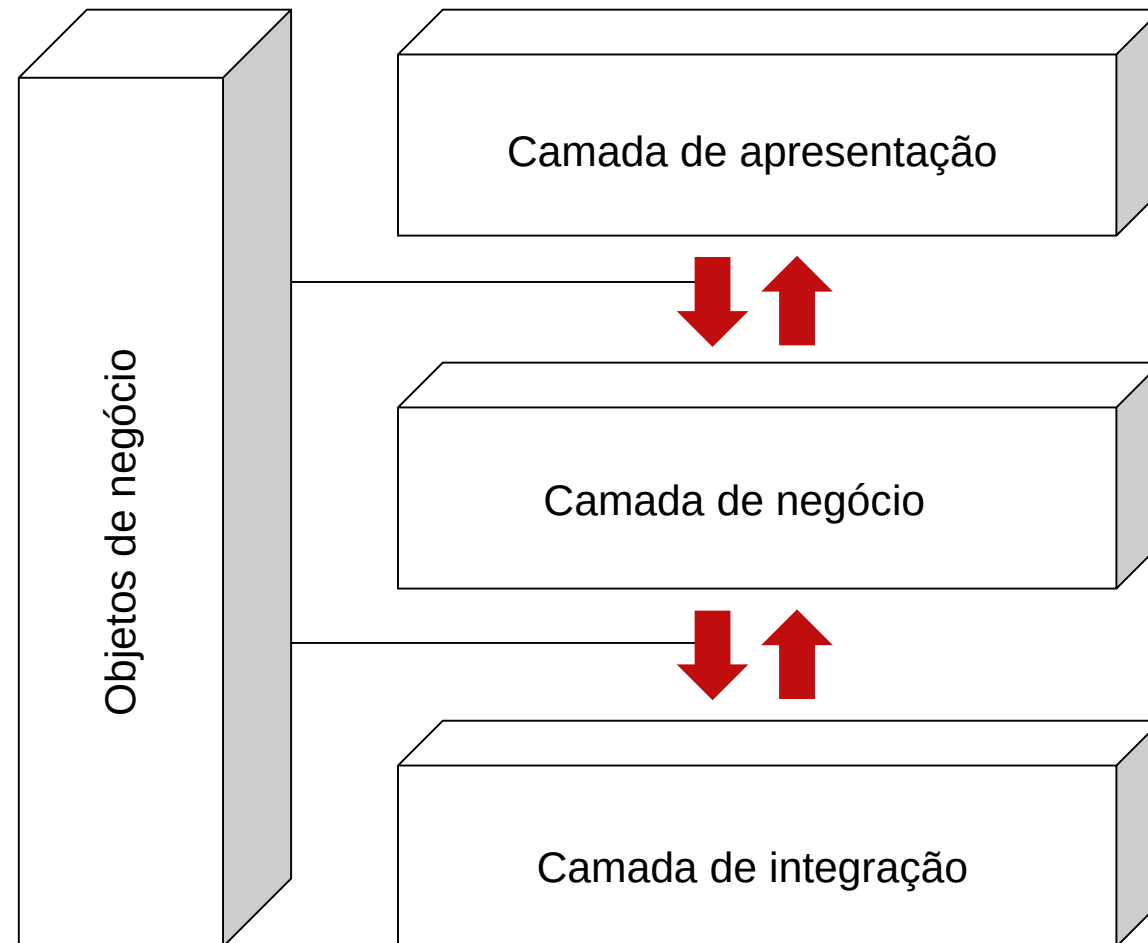
Solução

Nesta arquitetura, temos três camadas principais:

- **Camada de apresentação:** que contém classes responsáveis pela interação com o usuário, como páginas web, formulários etc.
- **Camada de negócios:** que contém classes responsáveis pela execução de regras de negócio, como cálculos, validações etc.
- **Camada de integração:** que contém classes responsáveis por fazer a integração com tecnologias externas ao sistema, como banco de dados, serviços web, além de outros sistemas ou dispositivos de hardware.

Capítulo 8 - Visões da arquitetura de software

Estruturação de sistemas em subsistemas e camadas:



Fonte: adaptado de: livro-texto.

Capítulo 8 - Visões da arquitetura de software

Neste caso, teríamos resolvido alguns problemas:

- **Divisão de responsabilidades:** cada “caixa” teria uma responsabilidade bem-definida.
- **Baixo acoplamento:** as “caixas” teriam apenas as dependências necessárias, por exemplo, as páginas web não precisariam depender do banco de dados e vice-versa.

Capítulo 8 - Visões da arquitetura de software

- **Boa manutenibilidade:** agora é possível saber exatamente em que “caixa” se encontram os componentes responsáveis por fazer algum tipo de cálculo ou executar alguma regra de negócio, bem como é possível paralelizar as atividades, cada equipe cuida da manutenção de uma “caixa”: uma cuida do banco de dados, outra das páginas web e uma terceira das mudanças nas regras de negócio. O trabalho é mais produtivo.
- **Reúso:** os componentes podem ser reutilizáveis, ou seja, você pode simplesmente usar um componente da “caixa” regra de negócio de quantas páginas web você quiser, com a certeza de que o código implementado é o mesmo, sem ter de utilizar o artifício de copiar o código.

Interatividade

Assinale a alternativa que descreve a principal vantagem de aplicar padrões de projeto em uma arquitetura em camadas.

- a) Permitir que os dados sejam armazenados diretamente na camada de apresentação.
- b) Facilitar a comunicação entre todas as classes do sistema.
- c) Reduzir a complexidade do sistema, promovendo separação de responsabilidades e facilitando manutenção e testes.
- d) Tornar o código mais lento, porém mais seguro, ao restringir o acesso entre camadas.
- e) Eliminar a necessidade de testes unitários, pois os padrões garantem a ausência de erros.

Resposta

Assinale a alternativa que descreve a principal vantagem de aplicar padrões de projeto em uma arquitetura em camadas.

- a) Permitir que os dados sejam armazenados diretamente na camada de apresentação.
- b) Facilitar a comunicação entre todas as classes do sistema.
- c) Reduzir a complexidade do sistema, promovendo separação de responsabilidades e facilitando manutenção e testes.
- d) Tornar o código mais lento, porém mais seguro, ao restringir o acesso entre camadas.
- e) Eliminar a necessidade de testes unitários, pois os padrões garantem a ausência de erros.

Capítulo 8 - Visões da arquitetura de software

Índice: 4ª parte

Continuação: projeto arquitetural

- Vimos no capítulo anterior **arquitetura em camadas e algumas soluções para arquitetura.**

Capítulo 8 - Visões da arquitetura de software

Índice: 4ª parte

Neste capítulo, vamos abordar:

- **Introdução a padrões de projeto:** Design patterns.
- **Ter uma visão dinâmica:** visões da UML arquiteturais.

Capítulo 8 - Visões da arquitetura de software

Introdução a padrões de projeto design patterns

- Padrão de projeto, ou design patterns, é a menor estrutura arquitetural proveniente dos subsistemas de um sistema de software e do relacionamento entre eles.

Capítulo 8 - Visões da arquitetura de software

Definição:

- Design patterns, padrões de projeto, são soluções reutilizáveis para problemas comuns que ocorrem com frequência no desenvolvimento de software orientado a objetos.

Capítulo 8 - Visões da arquitetura de software

- Design patterns em arquitetura em camadas.

Arquitetura em camadas:

- **Organiza o sistema em camadas como:** Apresentação, Aplicação, Domínio e Persistência.

Capítulo 8 - Visões da arquitetura de software

Como os padrões se aplicam:

- **DAO (Data Access Object):** Isola a lógica de acesso a dados (camada de persistência).
- **MVC (Model-View-Controller):** Separa a interface, a lógica de negócio e o controle (camada de apresentação).
- **Service Layer:** Centraliza a lógica de negócios (camada de aplicação).

Capítulo 8 - Visões da arquitetura de software

Exemplo de visão padrão MVC:

Camada	Responsabilidade
Model	Dados e regras de negócio (User)
View	Exibição dos dados para o usuário (UserView)
Controller	Coordena a lógica entre Model e View (UserController)

Fonte: autoria própria.

Capítulo 8 - Visões da arquitetura de software

Exemplo de visão padrão MVC:

- Responsável pela apresentação dos dados (interface com o usuário).

```
python
```

```
# view.py
```

```
class UserView:  
    def show_user(self, user):  
        print(f"Nome: {user.name}")  
        print(f"E-mail: {user.email}")
```

Capítulo 8 - Visões da arquitetura de software

Exemplo de visão padrão MVC:

- Faz a mediação entre a View e o Model.

```
python
```

```
# controller.py
from model import User
from view import UserView

class UserController:
    def __init__(self):
        self.view = UserView()

    def create_user(self, name, email):
        user = User(name, email)
        self.view.show_user(user)
```

Fonte: autoria própria.

Capítulo 8 - Visões da arquitetura de software

Exemplo DAO em C# – Cadastro de usuários com SQLite:

Camada	Classe	Função
Modelo	User	Representa os dados do usuário
DAO	UserDAO	Acesso ao banco (inserir e listar usuários) usando SQLite
View/Controller	Program	Interação com o usuário via console; atua como interface e controle simples

Capítulo 8 - Visões da arquitetura de software

Exemplo DAO em C# – Cadastro de usuários com SQLite:

User.cs:

- Modelo da entidade
- Representa a entidade

chsharp

// User.cs

```
public class User
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Email { get; set; }
}
```

Capítulo 8 - Visões da arquitetura de software

Exemplo DAO em C# – Cadastro de usuários com SQLite:

- **Classe DAO:** com acesso ao banco.
- Manipula banco de dados (CRUD)

```
chsharp

// UserDAO.cs
using Microsoft.Data.Sqlite;
using System.Collections.Generic;

public class UserDAO
{
    private readonly string _connectionString = "Data Source=users.db";

    public UserDAO()
    {
        using var connection = new SqliteConnection(_connectionString);
        connection.Open();

        var command = connection.CreateCommand();
        command.CommandText =

            @"
            CREATE TABLE IF NOT EXISTS Users (
                Id INTEGER PRIMARY KEY AUTOINCREMENT,
                Name TEXT NOT NULL,
                Email TEXT NOT NULL
            );
        ";
    }
}
```

Fonte: autoria própria.

Capítulo 8 - Visões da arquitetura de software

Exemplo DAO em C# – Cadastro de usuários com SQLite:

Program.cs

Interface simples via console

```
chsharp

// Program.cs
using System;

class Program
{
    static void Main()
    {
        var dao = new UserDAO();

        while (true)
        {
            Console.WriteLine("\n--- Menu ---");
            Console.WriteLine("1. Cadastrar usuário");
            Console.WriteLine("2. Listar usuários");
            Console.WriteLine("3. Sair");
            Console.Write("Escolha uma opção: ");
            var option = Console.ReadLine();
        }
    }
}
```

Fonte: autoria própria.

Capítulo 8 - Visões da arquitetura de software

Visão dinâmica:

Segundo Bass, Clements e Kazman (2010), são três as visões arquiteturais:

- **Visão modular:** representa a visão do sistema em termos de unidade de implementação; essas unidades podem ser classes, componentes ou módulos.

Capítulo 8 - Visões da arquitetura de software

Segundo Bass, Clements e Kazman (2010), são três as visões arquiteturais:

- **Visão componente e conector:** representa a forma pela qual os componentes interagem, ou seja, seus protocolos de comunicação.

Capítulo 8 - Visões da arquitetura de software

Segundo Bass, Clements e Kazman (2010), são três as visões arquiteturais:

- **Visão de alocação:** representa a forma pela qual esses componentes estão distribuídos em uma infraestrutura.

Capítulo 8 - Visões da arquitetura de software

Visões da UML arquiteturais

Visão arquitetural	Diagrama UML
Visão modular	Diagrama de pacotes
	Diagrama de classes
Visão componente e conector	Diagrama de componentes
Visão de alocação	Diagrama de distribuição

Capítulo 8 - Visões da arquitetura de software

Resumindo:

- A identificação dos elementos do projeto e de seus relacionamentos passa pela representação das duas visões da arquitetura de software: **a visão estática e a visão dinâmica.**

Capítulo 8 - Visões da arquitetura de software

Resumindo:

- Na visão estática, estruturamos o sistema em subsistemas e organizamos seus componentes internos; para isso, utilizamos uma ferramenta muito importante: os padrões.
- Estilos arquiteturais e padrões de projetos são importantes soluções para um problema específico em um contexto específico, e essas duas variáveis são importantes nessa hora. Não devemos usar esses padrões simplesmente por usar.

Capítulo 8 - Visões da arquitetura de software

Resumindo:

- Na visão dinâmica, modelamos como os componentes e os objetos se comunicam para resolver um determinado problema.
- Ambas as visões são registradas na documentação da arquitetura a partir de um conjunto específico de diagramas da UML.

Interatividade

A arquitetura de software é importante para:

- a) o funcionamento do sistema.
- b) estruturar o sistema em subsistemas e organizar seus componentes.
- c) modelar como os componentes e os objetos se comunicam.
- d) desenvolver o projeto para construção.
- e) modelar e colocar em evidência as decisões de projeto como fator fundamental para o sucesso do sistema de software.

Resposta

A arquitetura de software é importante para:

- a) o funcionamento do sistema.
- b) estruturar o sistema em subsistemas e organizar seus componentes.
- c) modelar como os componentes e os objetos se comunicam.
- d) desenvolver o projeto para construção.
- e) modelar e colocar em evidência as decisões de projeto como fator fundamental para o sucesso do sistema de software.

Referências

- ALBIN, S. T. *The art of software architecture: design methods and techniques*. Indianapolis: John Wiley, 2003.
- BASS, L.; CLEMENTS, P.; KAZMAN, R. *Software architecture in practice*. Boston: Addison-Wesley, 2010.
- ELMASRI, R.; NAVATHE, S. B. *Sistemas de banco de dados*. 6. ed. São Paulo: Addison-Wesley, 2011.
- OPENUP. *Introduction to OpenUP*. [s.d.]. Grupo de Gerenciamento de Objetos. Disponível em: <https://www.omg.org/> . Acesso em: 3 ago. 2025.
- PRESSMAN, R. S. *Engenharia de software*. 6. ed. São Paulo: Mcgraw-hill, 2006.
- SOMMERVILLE, I. *Engenharia de software*. 9. ed. São Paulo: Pearson, 2011.

ATÉ A PRÓXIMA!